

Audio Deepfake Detection — Project

Phase-by-Phase Execution Plan

From zero to a working, locally-run, professor-presentable, product-grade voice spoof detector

WHAT YOU TOLD ME AT THE START (from our first conversation)

- University research project (under a professor)
- Signal/speech processing + ML combined
- Product-grade, not just academic
- Audio deepfake / voice spoof detection
- Generalization to real-world / in-the-wild audio
- Local on-device deployment preferred
- Small team (ideation phase at start)
- Open + reproducible study as differentiator

0

SETUP & FIRST INFERENCE

Week 1

GOAL: get a working inference on one audio file — confirm the stack runs on your machine

DELIVERABLES

- ✓ conda env with PyTorch+torchaudio
- ✓ clovaai/aasist repo cloned
- ✓ pretrained AASIST-L weights loaded
- ✓ one file -> real/fake score in terminal

TASKS

- pip install torch torchaudio numpy soundfile
- git clone github.com/clovaai/aasist
- copy models/weights/AASIST-L.pth locally
- write 40-line infer.py wrapper
- test on 3 known real + 3 known fake files

Source: clovaai/aasist (NOT HuggingFace). Pretrained weights already in the repo under models/weights/.

1

SIGNAL PROCESSING CORE

Week 2-3

GOAL: build the shared preprocessing module — the golden-rule module used by both train and infer

DELIVERABLES

- ✓ preprocessing.py shared module
- ✓ handles any input format/rate/length
- ✓ CMVN stats file saved from training set
- ✓ unit tests: train vs infer output identical

TASKS

- decode: torchaudio.load() any format
- resample to 16kHz (polyphase resampler)
- mono downmix + amplitude normalize
- fix length: pad/crop to 64600 samples
- write test: same file -> same tensor both sides
- document every choice (rate, norm method)

GOLDEN RULE: every decision here locks in both sides. Write a test that asserts tensor equality before moving on.

2

TRAINING PIPELINE

Week 3-5

GOAL: train AASIST-L from scratch on ASVspoof 2019 LA — get your own weights, not borrowed ones

DELIVERABLES

- ✓ ASVspoof 2019 LA downloaded (~11GB)
- ✓ DataLoader for train/dev/eval splits
- ✓ train.py with LMCL loss + Adam
- ✓ best checkpoint saved (EER on dev set)
- ✓ training curve plot (loss vs epoch)

TASKS

- download dataset: asvspoof.org (Edinburgh DS)
- build dataset class + DataLoader
- RawBoost augment (convolutive+impulsive)
- copy AASIST-L model class from clovaai repo
- train loop: forward, loss, backprop, save best
- log EER on dev set each epoch
- pick best checkpoint by min dev EER

RawBoost: github.com/TakHemlata/RawBoost. Match augmentation to deployment: conv+impulsive for LA, stationary noise for DF.

3

EVALUATION & BASELINE EER

Week 5-6

GOAL: report your first real EER numbers on ASVspoof 2019 LA eval set — the in-domain baseline

DELIVERABLES

- ✓ eval.py: score every file, compute EER
- ✓ min t-DCF (official metric) reported
- ✓ comparison table vs published AASIST-L EER
- ✓ score distribution plots (real vs fake)

TASKS

- run inference on full eval set
- compute EER using official scoring script
- compute min-tDCF (in clovaai repo)
- plot score histograms for both classes
- confirm your number vs paper (expect ~0.83%)
- flag any gap — investigate if >2x off

Target: AASIST-L paper = 0.83% EER on ASVspoof 2019 LA. If much worse, gap is in preprocessing or data loading.

4

CROSS-DOMAIN STRESS TEST

Week 6-7

GOAL: quantify the generalization gap — test your 2019-trained model on ASVspoof 2021 DF (codec-degraded)

DELIVERABLES

- ✓ ASVspoof 2021 DF eval scores
- ✓ EER comparison table: 2019 LA vs 2021 DF
- ✓ degradation number (the research finding)
- ✓ per-codec breakdown if possible

TASKS

- download ASVspoof 2021 DF eval partition
- run same frozen model on 2021 DF (no retraining)
- compute EER + tDCF on 2021 DF
- build comparison table (expect ~15% EER vs ~0.8%)
- test with Opus/AMR codec simulation too
- write findings section: THIS is your contribution

Expected: EER jumps ~0.8% -> ~15%. That 20x degradation IS the finding. Do not fix it - measuring it is Phase 4.

5

CALIBRATION + ABSTAIN BAND

Week 7-8

GOAL: add your research contribution: calibrate the raw score into a 3-band verdict with an honest abstain zone

DELIVERABLES

- ✓ Platt calibration fitted on dev-set scores
- ✓ 3-band thresholds set at EER operating point
- ✓ abstain% reported: in-domain vs cross-domain
- ✓ result: how much error the abstain zone absorbs

TASKS

- collect raw scores on dev set
- fit LogisticRegression (Platt) on dev scores
- set two thresholds on calibrated probability
- run on 2019 eval: measure per-band distribution
- run on 2021 DF eval: measure per-band distribution
- report: X% of cross-domain errors -> abstain
- this comparison IS the paper-quality finding

Your differentiator. No commercial tool publishes abstain-zone characterization. Abstain band absorbing cross-domain error = novel, citable finding.

6

EXPLAINABILITY + WEB UI

Week 8-10

GOAL: make the system legible to a non-expert: Grad-CAM on the spectrogram + a simple upload-and-score web page

DELIVERABLES

- ✓ Grad-CAM overlay on spectrogram (PNG out)
- ✓ per-segment timeline plot
- ✓ FastAPI endpoint: POST audio -> JSON verdict
- ✓ simple web page: upload -> result card
- ✓ live mic demo (optional but impressive)

TASKS

- implement Grad-CAM on AASIST-L final layer
- generate spectrogram + overlay heatmap
- build per-window scorer + timeline plot
- wrap inference in FastAPI POST /analyze
- return JSON: band, prob, segments, heatmap_png
- build minimal React or plain HTML frontend
- test end-to-end: browser upload -> verdict

Stack: FastAPI + uvicorn (backend), React or plain HTML (frontend). ONNX export for 2-5x faster CPU inference. No cloud needed.

7

CODEC ROBUSTNESS STUDY

Week 10-11

GOAL: measure how your detector degrades under real-world codecs and whether RawBoost augmentation closes the gap

DELIVERABLES

- ✓ codec simulation pipeline (Opus/AMR/MP3)
- ✓ EER vs bitrate table for 3+ codecs
- ✓ RawBoost vs no-RawBoost comparison
- ✓ conclusion: which codec kills detection most

TASKS

- re-encode eval files at 3 bitrates (Opus 8/16/32kbps)
- re-encode at AMR-NB 12.2kbps (phone quality)
- run scorer on each re-encoded set
- retrain with stationary noise RawBoost for DF
- compare EER: original vs codec-degraded vs augmented
- write findings: codec-robustness is the open problem

Tool: ffmpeg for re-encoding. This is the Codecfake open problem (2024/2025 papers). Which codec kills detection most = your finding.

WHAT MAKES THIS PRODUCT-GRADE, NOT JUST ACADEMIC

- open + reproducible (published architecture, generalization on unseen + abstain (honest) band lives) — not a commercial product (330KB model, no API, runs on a laptop — laptop possible)
- the finding that matters: how and

GOAL: package everything into a defensible, presentable research deliverable

DELIVERABLES

- ✓ written report (intro, method, results, discussion)
- ✓ final architecture panel (already done)
- ✓ results tables: EER, tDCF, abstain%, codec
- ✓ live demo: one-command run on a new file
- ✓ wav2vec2 upper-bound reference (1 experiment)

TASKS

- write related work (4 papers + generalization lit)
- write method section from the architecture panel
- compile all result tables into one findings sheet
- record a short demo video (upload -> verdict)
- run wav2vec2+AASIST once: report vs AASIST-L
- prepare 5-min pitch: problem, system, finding

Pitch: (1) real-world problem, (2) our lightweight local system, (3) the finding - abstain absorbs X% cross-domain error, (4) roadmap.